

Certified Inference: Pre-Registered, Kernel-Checked Certification of a Pretrained Language Model

Brian Naughton

Cambridge, UK · [linkedin.com/in/bnaughton](https://www.linkedin.com/in/bnaughton) · github.com/brian-naughton

4 July 2026

Abstract

We certify next-token predictions of a real pretrained language model — TinyStories-1M, an eight-layer GPT-Neo we neither trained nor modified — and wrap the per-sample certificates in a pre-registered statistical claim. The certified object throughout is the *exact-real transformer*: the function obtained by reading the released float32 tensors as exact dyadic rationals, with exact real arithmetic carried through every LayerNorm, GELU, and softmax, and the next-token argmax taken over exact real logits. We do not certify any binary32 PyTorch execution, whose operational semantics we leave unformalised. On a pre-registered with-replacement sample of $n = 1000$ prompts from a pinned finite token-id corpus C at context length 16, every sampled prompt had a *unique* full-vocabulary (50,257-token) exact-real next-token argmax certified at precision $P \leq 256$, by fixed-point interval arithmetic with outward rounding and re-derived by a torch-free checker sharing an audited arithmetic core. By the stated one-sided Hoeffding bound — kernel-checked in Lean 4 with Mathlib — with confidence $\geq 97.5\%$, at least 95.70% of C is certifiable under this engine and escalation policy. In the same frozen run the pinned float32 harness agreed with the certified argmax on all 1000 samples, giving a separate joint-event lower bound under a passed determinism gate. A single GPT-2-small certificate is reported as prestige confirmation only. The claim quantifies over the pinned finite corpus C alone; the trust boundary and its failure modes are in §6.

1 Introduction

This note documents an artifact: an independently re-checkable certificate chain over the next-token predictions of a real pretrained language model, wrapped in a pre-registered statistical bound. The model is **TinyStories-1M** [1], an eight-layer GPT-Neo ($d_{\text{model}} = 64$, 16 heads, MLP width 256, pre-LayerNorm with $\varepsilon = 10^{-5}$, the `gelu_new` tanh-approximate activation, learned positional embeddings, tied unembedding, vocabulary 50,257), taken from its published float32 checkpoint, sha-pinned and unmodified. We did not train it; we certify a formalisation of what it computes.

The certified object is the *exact-real transformer* — the released tensors read as exact dyadic rationals, with exact real matrix sums, an exact interval treatment of LayerNorm, GELU and softmax, and the argmax over exact real logits. The authors' PyTorch execution (binary32, kernel-dependent reductions) is a *conformance target*, *not the theorem*; this framing recurs throughout, because it fixes what the certificates do and do not say.

Two properties are certified per sample. φ_1 , the scientific core, is that the exact-real next-token argmax is *unique* over the full 50,257-token vocabulary at the certified precision. $\varphi_{2,\text{joint}}$ is that a pinned float32 inference harness agrees with that certified argmax. These per-sample facts are then lifted to a finite population by a pre-registered Hoeffding bound over a pinned token-id corpus C . The bound quantifies over C — a fixed, sha-pinned list of windows at one context length — and over nothing broader: not TinyStories validation, not story prompts generally, and not any deployment distribution. Table 1 summarises the claim status; the population language attaches to C everywhere it appears.

This is the third in a trilogy. *verified-circuits* [9] established the interval method on a self-trained model with a length-generic Lean theorem and an exhaustive finite-domain certificate; *certified-grokking* [10] certified the published Nanda et al. [7] grokking checkpoint over its entire 113^2 -input domain, again a model with *no* LayerNorm by design. The step here is distinct: leave finite enumeration behind, add LayerNorm and GELU, certify full-vocabulary next-token argmax on a real pretrained language

Claim	Current status
φ_1 : unique full-vocabulary exact-real next-token argmax on every sampled ctx-16 prompt	Certified, re-checkable
Hoeffding bound: $\geq 95.70\%$ of the pinned corpus C , confidence $\geq 97.5\%$	Kernel-checked (Lean 4)
$\varphi_{2,\text{joint}}$: pinned float32 harness agrees with the certified argmax	Conformance; gate passed
GPT-2-small full-vocabulary next-token certificate ($n = 8$)	Confirmation only

Table 1: Status of the certified claims. φ_1 carries the title. The statistical lower-bound theorem is kernel-checked in Lean; per-sample facts are interval-certified with outward rounding and torch-free re-checked; sampling and harness provenance are audited. The checkpoint and corpus are provenance-pinned by sha256.

model, and lift the per-sample facts to a statistical claim. This note does not pretend to the same formal closure as the first: the population claim rests on a statistical wrapper over audited certificate records, not on end-to-end enumeration.

2 Related work

Robustness verification for transformers was opened by Shi et al. [2], who propagate linear bounds through self-attention over an input region. That region-based lineage — and the finite-domain interpretability certificates of the trilogy’s first two entries — typically avoids or deletes LayerNorm, whose mean/variance normalisation induces severe dependency blow-up under interval or linear-relaxation domains; we instead keep LayerNorm and walk the enclosure through it (§3.2). Somani [3] converts task-localised circuits into solver-checkable, SMT-amenable claims, an architecture-modifying, discovery-oriented route distinct from per-sample exact-real functional certification; Anani et al. [4] certify *stability* of the circuit-discovery procedure rather than per-sample output correctness. TorchLean [5] formalises neural networks in Lean 4 so specification, execution, and verification share one semantics — the gold-standard answer to the exact-real/binary32 gap our harness only measures. Szász, Bánhelyi & Jelasity [6] document why sound verification claims break on *deployed* networks — floating point, hardware, library nondeterminism — precisely the boundary we refuse to cross. The present note contributes an exact-real, full-vocabulary next-token certificate for a real LayerNorm language model, plus a kernel-checked statistical wrapper.

3 Method

3.1 The certified object

The certified object is the exact-real transformer: the function

$$f: w \longmapsto \arg \max_{c \in \{0, \dots, 50256\}} \text{logit}_c(w),$$

mapping a token-id window w to its next-token class, where every logit_c is evaluated by reading the published float32 tensors as *exact dyadic rationals* and carrying exact real arithmetic — matrix sums, LayerNorm, GELU, softmax — to the final logits, with the argmax over exact reals. Each weight is exact: a float32 value has a ≤ 24 -bit mantissa, so its widening to Python float (float64) is lossless. φ_1 asserts the argmax is *unique* — the target logit strictly exceeds every one of the other 50,256. The PyTorch forward pass is never the theorem.

3.2 The interval engine, and walking through LayerNorm

Enclosures are computed in the certified-grokking fixed-point interval core (dyadic integer endpoints, *outward rounding* throughout), extended here with the three primitives a real language model needs: a rigorous interval LayerNorm (interval mean and variance, and $1/\sqrt{\cdot}$ over an interval), `gelu_new` and `tanh` via a rigorous `exp` enclosure, and a max-shifted interval softmax. The certified statement

reduces, as in the finite-domain work, to a lower bound on the target logit exceeding an upper bound on every competitor.

The literature that certifies over input *regions* deletes or approximates LayerNorm because the variance division loses dependency information catastrophically across a box of inputs. We instead certify a *point* input — a single prompt — at high precision. On a point, no input width enters; the only enclosure width is the outward-rounding residue of the arithmetic itself, and that shrinks as the working precision P rises. This is what makes it feasible to walk an eight-layer LayerNorm chain end to end rather than route around it.

Empirically the residue compounds multiplicatively with depth. On TinyStories-1M the residual-stream enclosure width grows by a precision-independent geometric-mean factor of $\approx 2^{17.3}$ per layer — the two LayerNorms dominate it ($\approx 200\text{--}300\times$ each, from the variance division’s dependency loss), softmax adds $\approx 20\times$, and the matmuls and GELU are roughly width-neutral. Because the factor is precision-independent, the certified bit-width behaves like $P - 17.3L$ after L layers and roughly *doubles* as $P \rightarrow 2P$: a clean linear-in-precision wall. At $P = 96$ the widths cross $O(1)$ mid-network and the final LayerNorm explodes; at $P = 192$ the full chain survives with final-logit width $\sim 10^{-10}\text{--}10^{-12}$ against decision gaps of order one, 30–40 bits of headroom. GPT-2-small (12 layers) shows the same mechanism at a steeper slope ($\approx 2^{23.6}$ per layer).

3.3 Six bugs found, and the standing canary

The interval extensions passed through two rounds of adversarial review; four bugs were found and fixed, all as strict tightenings (never loosening a sound bound), with regression tests added. Two were guard fast-path floors: an `exp` underflow shortcut sound only up to $P \approx 101$ and a `tanh` saturation shortcut sound only up to $P \approx 137$, each returning an interval too narrow at higher precision. Both original certificates were *accidentally* sound — audit logging (added with the fix) shows neither fast path ever fired on the decision path — and both are now precision-parametric, guarded by a threshold derived from a rational upper bound on $\ln 2$. The other two were precision-independent floors: an `exp` truncation tail whose term count did not grow with P , and a hard-coded 61-digit- π constant that capped every GELU at ≈ 200 effective bits (it aborted the first GPT-2 run mid-layer-0), replaced by a rigorous rational π enclosure via Machin’s identity to a precision-scaled width. A *widths-halve-when-P-doubles* canary — assert, for every sublayer, that the enclosure at $2P$ is far tighter than at P — caught two of these four and is now a standing gate: a precision-independent floor is exactly a sublayer that fails it.

Two further hardening fixes preceded the freeze. First, a GPT-2 partial-vocabulary bug in which the certified competitor set could silently downgrade from the full 50,257 tokens to a top-200 shortlist; the dispatch now carries a `require_full` guard that *raises rather than degrades*, so a partial competitor set aborts rather than certifies. Second, the confidence budget δ and its split are now carried as exact numerator/denominator pairs parsed to rationals with an exact sum-check, replacing float δ and a 10^{-12} tolerance — no floating-point comparison anywhere on the certification path.

3.4 The two-instrument trust boundary

Certificates are generated by one instrument and re-derived by a second. The loader sha-verifies the pinned `pytorch_model.bin` on every call and exports each named tensor as `float.hex()` strings, torch-free downstream; the checker (`certificates/check.py`) re-computes every claimed quantity from those hex weights and the committed token-id corpus *without* PyTorch, mirroring the transformer wiring separately and requiring bit-identical agreement, trusting the certificate for nothing except which inputs it claims. For a headline (pre-registered) certificate it also asserts full-corpus completeness against the frozen sample multiset, so no sample can be silently omitted or an extra one slipped in.

The honest description is a *torch-free checker sharing an audited arithmetic core* — not two independent implementations. The checker is independent of PyTorch and of the generator’s weight-loading path, but imports the same `certinf.exact` and `certinf.ival_ext`: a soundness bug in outward rounding, the `exp` enclosure, LayerNorm intervalisation, the GELU/tanh guards, or the softmax logic would fool both instruments. “Bit-identical” therefore means *determinism plus the soundness of that shared*

core, and the strongest upgrade is an external arithmetic audit or a second, genuinely independent implementation of the primitives.

3.5 Pre-registration: what the witness proves, and what it does not

The population claim is pre-registered. Before any headline certificate exists, a freeze commit fixes the corpus, the checkpoint, the sample size $n = 1000$, the confidence budget $\delta = \frac{1}{20}$ split $\frac{1}{40} + \frac{1}{40}$, the precision ceiling $P_{\max} = 256$, and a with-replacement draw of sample indices. The seed is *nothing-up-my-sleeve*: the project date, 20260703, declared as such. Draws are with replacement and duplicates are kept, counted with multiplicity and never deduplicated — because the Hoeffding wrapper’s independence hypothesis holds for i.i.d. draws *with* replacement, and deduplication would break it.

The witness `prereg.verify()` proves the committed `prereg.json` and `sample-index.json` are the deterministic, arithmetically-consistent output of the committed seed against the committed corpus. It does *not* prove the seed was chosen before the results were seen: a `prereg.json` can be regenerated after the fact and still verify `True`. Pre-commitment is established *externally* — by the freeze commit’s public timestamp, containing the bound parameters and *no* certificate results, and by publishing the pre-registration hash as each record’s `prereg_ref`. The version-control timestamp is the defence; the witness is only its determinism. There is no adaptive stopping and no n -extension: n is fixed at the freeze and the run carried to it regardless of interim observations.

3.6 The Hoeffding wrapper

The per-sample indicators are lifted to a finite-population rate by a one-sided Hoeffding bound, stated and proved in Lean 4 with Mathlib (`proofs/HoeffdingWrapper/Basic.lean`).

Theorem 1 (`hoeffding_lower_confidence_count`). *Let $B_0, \dots, B_{n-1}$ be independent $[0, 1]$ -valued random variables (in particular $\{0, 1\}$ -indicators) on a probability space with common mean $\mu[B_i] = p$. Then with probability at least $1 - \delta$,*

$$p \geq \frac{1}{n} \sum_i B_i - \sqrt{\frac{\ln(1/\delta)}{2n}},$$

so any certified success count $k \leq \sum_i B_i$ yields $p \geq k/n - \sqrt{\ln(1/\delta)/(2n)}$.

The proof routes each centred $B_i - p$ through Hoeffding’s lemma to a sub-Gaussian parameter $c = 1/4$; the sub-Gaussian Chernoff bound for independent sums then gives $\Pr[\sum (B_i - p) \geq n\varepsilon] \leq \exp(-2n\varepsilon^2)$, which equals δ at $\varepsilon = \sqrt{\ln(1/\delta)/(2n)}$. It is mechanised with zero sorries and axioms exactly [`propext`, `Classical.choice`, `Quot.sound`].

One anecdote earns the kernel its keep. The task sketch handed to the formalisation had the tail the wrong way round — it controlled the upper tail of $\sum (p - B_i)$, which bounds the *upper* confidence bound $p \leq k'/n + \varepsilon$, not the lower one the headline needs. The failure mode of $p \geq k/n - \varepsilon$ is the empirical mean *overshooting* p , i.e. the upper tail of $\sum (B_i - p)$. Both tails obey the same bound by symmetry, so a paper proof waves the direction through; the Lean type-checker did not, and surfaced the flip. The finite-corpus sampling bridge — that seeded with-replacement draws from C instantiate the theorem’s hypotheses — is deliberately *not* formalised; it is discharged by audited provenance, and the kernel-checked object is the statistical inequality itself.

4 Results

Headline (φ_1). The pre-registered run certified 1000/1000 sampled ctx-16 prompts: every sample had a unique full-vocabulary exact-real next-token argmax, at $P \leq 256$ (in fact the escalation ladder $P \in \{128, 160, 192\}$ never exceeded 192), with zero abstentions. The independent torch-free checker re-derived all 1000 records from the hex weight export with the completeness assertion against the frozen multiset, printing verbatim:

```
VERIFIED (1000 records re-derived from hex weights, all records, headline)
```

With $k = n = 1000$ and $\delta_1 = \frac{1}{40}$, the one-sided Hoeffding lower bound is $L_1 = 1 - \sqrt{\ln(40)/2000} = 1 - 0.042947 = 0.957053$, displayed rounded *down* to a headline of **at least** 95.70% of the pinned finite corpus C certifiable, at confidence $\geq 97.5\%$. The ε display is rounded *up* so the lower bound can only round down — the display never overstates the rate.

Harness agreement ($\varphi_{2,\text{joint}}$). In the same frozen run the pinned float32 harness’s top-1 token agreed with the certified exact-real argmax on all 1000 samples ($k_2 = 1000$), giving a separate lower bound $L_2 = 0.957053$ on the joint event “certified *and* harness-agrees” at confidence $\geq 97.5\%$. The determinism gate *passed*: under the exact frozen command line at ctx-16, five repetitions produced bit-identical top-1 tokens and transcripts. This is pinned-environment conformance — an implementation transcript (hardware/OS/Python/PyTorch/flags/checkpoint/command line), not a binary32 operational semantics.

Calibration evidence (non-headline). A separate calibration grid (every record `prereg_ref = None`; calibration samples are never headline samples) certified 300/300 across three context lengths with zero abstentions (Table 2). Required precision is tight and not heavy-tailed: p95 = 160 everywhere, p99 tracking p95. This is a calibration table, not a precision–depth law: the fit degenerates to a single family’s point and estimates no cross-family slope.

model	ctx	n	n_{cert}	required- P p95	required- P p99	abstentions
tinystories	8	100	100	160	160.0	none
tinystories	16	100	100	160	160.3	none
tinystories	32	100	100	160	160.0	none

Table 2: Phase 1 calibration grid ($P_{\text{max}} = 256$), non-headline. All 300 records were re-derived bit-identically by the torch-free checker. Required- P is the escalation rung at which each sample certified; the single p99 excursion at ctx-16 is one sample certifying at 192.

GPT-2-small confirmation (no population claim). As prestige confirmation of scale — not a statistical headline — $n = 8$ full-vocabulary exact-real next-token certificates were produced for GPT-2-small on a deterministic prefix slice of a pinned WikiText-103 corpus at ctx-16: 8/8 certified and 8/8 harness-agreeing at the first ladder rung $P = 320$, zero abstentions, with 12.1–27.6 bits of per-sample headroom. These come from the *same* generator instrument (the `require_full` full-vocabulary path) but are *not* yet re-derived by a second torch-free instrument: the checker’s residual-stream wiring is TinyStories-shaped with no GPT-2 path yet. GPT-2 carries no Hoeffding bound and no population language; closing the checker path is future work.

5 Trust boundary and threat model

The trusted base is stratified. The statistical lower-bound theorem is *kernel-checked* (Lean plus its three axioms); the per-sample facts are *interval-certified* with outward rounding and torch-free re-checked; the sampling and harness provenance are *audited* (the freeze commit’s timestamp, the pre-registration hash in every record, the corpus and checkpoint sha256 pins, the recorded determinism transcript). The wrapper is a kernel-checked statistical wrapper over audited certificate records — never “certified end-to-end”.

Seven ways this could still be wrong, named rather than hidden. (i) A *shared-core bug* would fool both instruments, since generator and checker import the same interval math; the core is small, hand-written, adversarially reviewed, and offered for direct review, but torch-free re-derivation is not an independent implementation. (ii) The *exact-real vs binary32* gap: the harness is an implementation transcript, its agreement gated conformance, never a theorem about binary32. (iii) The claim quantifies over the *pinned finite corpus C* only — a deterministic first-slice construction, not TinyStories validation or any deployment distribution. (iv) *Corpus representativeness*: the tight precision profile could be an artefact of the first slice; a randomised-corpus table is on the agenda. (v) The pre-registration *witness is determinism, not pre-commitment*; the git timestamp is the real defence. (vi) *Lean status*: the kernel checks the statistics; the per-sample facts remain interval-certified and

the sampling audited. (vii) The *checkpoint/export boundary*: the pinned artifact is trusted to be the published checkpoint, and the float→rational export is corroborated, not kernel-checked. Finding an actual instance of any of these is exactly the peer review being requested.

6 Limitations and strengthening agenda

Each limit names its next step. (i) *Shared core*. A second, genuinely independent implementation of the interval primitives — or Lean-checked lemmas for the most dangerous ones (LayerNorm, softmax, GELU) — would turn agreement into cross-validation, worth more than more records through the same code. (ii) *GPT-2 checker path*. A GPT-2 residual-stream path in the checker would give the confirmation certificates a second instrument. (iii) *Calibration breadth*. A randomised-corpus table and further model families and context lengths would show the p95= 160 profile is not a first-slice artefact and begin to estimate a genuine cross-family slope. (iv) *Arithmetic domains*. Affine arithmetic or zonotopes around the two LayerNorms would reduce the width blow-up if a heavier precision tail appears; Clopper–Pearson or empirical-Bernstein wrappers would sharpen the bound.

7 Reproducibility

The repository ships a trust ladder of increasing cost and coverage: (1) stdlib tests — no downloads, no Torch — with the widths-halve canary and fixture re-derivations; (2) the pre-registration dry run, confirming the checker rejects a dropped, an extra, and a mixed-reference record; (3) the kernel-checked Lean proof with its axiom audit; (4) sampled headline re-checks; and (5) the full headline lane — fetch the sha-pinned checkpoint, run all 1000 certifications, and re-derive every record torch-free with completeness, reproducing the verbatim checker line above. All artifacts and commands are at github.com/brian-naughton/certified-inference.

Acknowledgements

TinyStories-1M is due to Eldan & Li; the interval method is adapted from this project’s two predecessors. This project was executed AI-first — Claude (Anthropic) served as researcher-engineer in an agentic loop, with GPT-5.5 (OpenAI Codex) for standing adversarial review at each design gate — while direction, judgement, and responsibility rest with the author. Peer review is genuinely requested, especially on the shared arithmetic core, the exact-real semantics, the LayerNorm and softmax intervalisation, and the pre-registration discipline. Corrections and failed replications are welcome.

References

- [1] R. Eldan & Y. Li. *TinyStories: How small can language models be and still speak coherent English?* arXiv:2305.07759.
- [2] Z. Shi, H. Zhang, K.-W. Chang, M. Huang & C.-J. Hsieh. *Robustness verification for transformers*. ICLR 2020. arXiv:2002.06622.
- [3] N. Somani. *Towards verifiable transformers: solver-checkable circuit explanations*. arXiv:2605.24033.
- [4] A. Anani, T. Lorenz, B. Schiele, M. Fritz & J. Fischer. *Certified circuits: stability guarantees for mechanistic circuits*. ICML 2026. arXiv:2602.22968.
- [5] R. J. George, J. Cruden, W. Adkisson, X. Zhong, H. Zhang & A. Anandkumar. *TorchLean: formalizing neural networks in Lean*. arXiv:2602.22631.
- [6] A. Szász, B. Bánhelyi & M. Jelasity. *No soundness in the real world: on the challenges of the verification of deployed neural networks*. ICML 2025. arXiv:2506.01054.
- [7] N. Nanda, L. Chan, T. Lieberum, J. Smith & J. Steinhardt. *Progress measures for grokking via mechanistic interpretability*. ICLR 2023. arXiv:2301.05217.
- [8] W. Hoeffding. *Probability inequalities for sums of bounded random variables*. *Journal of the American Statistical Association* **58**(301), 13–30, 1963.
- [9] B. Naughton. *verified-circuits*. <https://github.com/brian-naughton/verified-circuits>.
- [10] B. Naughton. *certified-grokking*. <https://github.com/brian-naughton/certified-grokking>.